1 / 8

Conversion happens before the products.

A smaller input format can round a value before accumulation.

> Inputs first enter FP32 storage; display values are rounded.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

Every row of A has 16 FP32 inputs

First 8: 1.0003 · last 8: -1.0

B contains only 1.0

256 outputs share this row-sum example

FP32 input → smaller format → product → FP32 sum

Displayed values are rounded; calculations use model values.

Copyright © 2026 Christopher Snow

2 / 8

The small excess can disappear.

Both smaller formats round these near-one inputs to 1.0.

> Conversion loss occurs before accumulation.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

FP16

1.0003 → 1.0 · -1.0 stays -1.0

Changed row inputs: 8 / 16

BF16

1.0003 → 1.0 · -1.0 stays -1.0

Changed row inputs: 8 / 16

FP32

1.0003 → 1.0003 · -1.0 stays -1.0

Changed row inputs: 0 / 16

Copyright © 2026 Christopher Snow

3 / 8

Compare two FP32 running sums.

Use the same order for converted and original inputs.

> Both paths round every addition to FP32.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

After K = 7: 8 additions into each FP32 sum

Converted inputs

Selected input × B: 1.0 × 1.0

Running sum: 8.0

Original FP32 inputs · same order

Selected input × B: 1.0003 × 1.0

Reference sum: 8.0024

The next eight inputs are −1.0.

Copyright © 2026 Christopher Snow

4 / 8

Cancellation exposes the lost excess.

Eight negative ones remove most of the positive sum.

> Changing the print format cannot restore discarded input data.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

After K = 15: 16 additions into each FP32 sum

Converted inputs

Selected input × B: -1.0 × 1.0

Running sum: 0.0

Original FP32 inputs · same order

Selected input × B: -1.0 × 1.0

Reference sum: 0.0024

The final difference originates in input conversion.

Copyright © 2026 Christopher Snow

5 / 8

Range is a separate question.

Very small positive inputs behave differently in FP16 and BF16.

> Range and nearby-value precision are different properties.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

16 small inputs: each begins as 1.00000e-8 in FP32

FP16

Converted input: 0.0

Final FP32 sum: 0.0

BF16

Converted input: 1.00117e-8

Final FP32 sum: 1.60187e-7

BF16 values stay non-zero here, but still round.

Nearest-even input conversion · sequential FP32 additions · rounded display · no tensor-core execution

Copyright © 2026 Christopher Snow

6 / 8

Arithmetic work does not identify the hardware.

Tensor cores support selected matrix multiply–accumulate operations.

> No tensor-core execution or GPU timing is simulated.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

Arithmetic in this chosen matrix shape

16 × 16 outputs × 16 K positions

4,096 scalar multiply–accumulates

Hardware implementation

Supported matrix shapes and formats depend on the GPU.

The sequential browser trace does not identify hardware.

Arithmetic work, instruction count and GPU time are distinct.

Copyright © 2026 Christopher Snow

7 / 8

Pause: test exactly representable inputs.

Alternate 1.0 and −0.5 across sixteen inputs. B stays all ones.

> The source row is visible; conversion answers are withheld.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

A new row · B remains all 1.0

Alternate 1.0 and -0.5 · 16 inputs

Predict FP16 and BF16 conversion

Changed inputs: ?

Final sum: ?

Apply exact representability, then pause.

Copyright © 2026 Christopher Snow

8 / 8

Agreement applies to the tested inputs.

No inputs change; all three formats give 4.0 here.

> Use a reference and data representative of the real calculation.

A: 16 × 16 · B: 16 × 16 · C: 16 × 16

A new row · B remains all 1.0

Alternate 1.0 and -0.5 · 16 inputs

FP16 and BF16 keep these values

Changed inputs: 0 / 16

Converted sum: 4.0 · reference: 4.0

Agreement on one row does not validate all other inputs.

Copyright © 2026 Christopher Snow